

INSTITUTO TECNOLÓGICO DE VERACRUZ

Nombre de la Asignatura:

- Administración en Base de Datos

Grupo: **AD26 6J3A**

Nombres del alumno:

- Raúl Alberto Guerrero Aguilar
- Num. Control: 23600266

Nombre de la Actividad:

- Reporte de Práctica 4 y 5

Carrera: Ingeniería en Sistemas Computacionales

Fecha de Entrega: 07/Octubre/2026

1. Introducción Teórica

El **Principio de Mínimo Privilegio** (Principle of Least Privilege, PoLP) es un lineamiento fundamental de la seguridad de la información que establece que todo sujeto —usuario, proceso o aplicación— debe contar únicamente con los permisos estrictamente indispensables para realizar su función, y solo durante el tiempo necesario. Su aplicación reduce la superficie de ataque, limita el alcance de un error operativo o de una credencial comprometida y facilita la trazabilidad de las acciones realizadas sobre los activos de información.

El **Control de Acceso Basado en Roles** (Role-Based Access Control, RBAC) es el mecanismo mediante el cual se materializa dicho principio de manera escalable. En lugar de asignar permisos de forma individual a cada usuario, los privilegios se agrupan en **roles** que modelan funciones organizacionales concretas, y los usuarios heredan las capacidades de los roles que les son concedidos. Este esquema desacopla la identidad de la autorización, simplifica la administración, permite auditar con claridad quién puede hacer qué y reduce la probabilidad de asignaciones inconsistentes.

En una base de datos de grafos como **Neo4j**, la información sensible rara vez se concentra en una tabla o colección aislada: se distribuye en nodos, relaciones y propiedades fuertemente interconectados, de modo que una consulta de recorrido puede alcanzar datos confidenciales a partir de un punto de entrada aparentemente inocuo. Por ello, resulta crítico aplicar el control de acceso con granularidad de **etiqueta de nodo**: Neo4j permite conceder privilegios sobre conjuntos específicos de nodos (por ejemplo, aquellos etiquetados como **Movie**), de forma que el motor haga invisible el resto del grafo para el usuario restringido. Esta capacidad convierte al PoLP en una garantía aplicada por la propia base de datos y no en una simple convención del código de aplicación.

2. Entorno de Trabajo

Para el desarrollo de la práctica se emplearon las siguientes herramientas:

- **Neo4j Desktop:** entorno de administración local utilizado para crear y operar la instancia de la base de datos.
- **Neo4j Browser:** interfaz web interactiva para la ejecución de sentencias, el uso de comandos de cliente (:play, :use, :server) y la visualización de resultados y grafos.
- **Cypher:** lenguaje declarativo de consulta y administración de Neo4j, empleado tanto para la manipulación de datos como para la gestión de usuarios, roles y privilegios.

3. Metodología (Desarrollo Paso a Paso)

Paso 1. Población de la base de datos

Se inicializó el entorno con una base de datos local y se utilizó el comando interactivo `:play movies` para cargar el grafo de prueba estándar de Neo4j. Este conjunto de datos contiene nodos con las etiquetas `Movie` y `Person`, relacionados entre sí (por ejemplo, mediante `ACTED_IN` y `DIRECTED`), lo que ofrece un escenario idóneo para demostrar el aislamiento por etiqueta: los nodos `Movie` representarán la información de acceso permitido, mientras que los nodos `Person` representarán la información que debe permanecer protegida.

```
// Carga de la guía interactiva con el grafo de ejemplo (nodos Movie y Person)  
:play movies
```

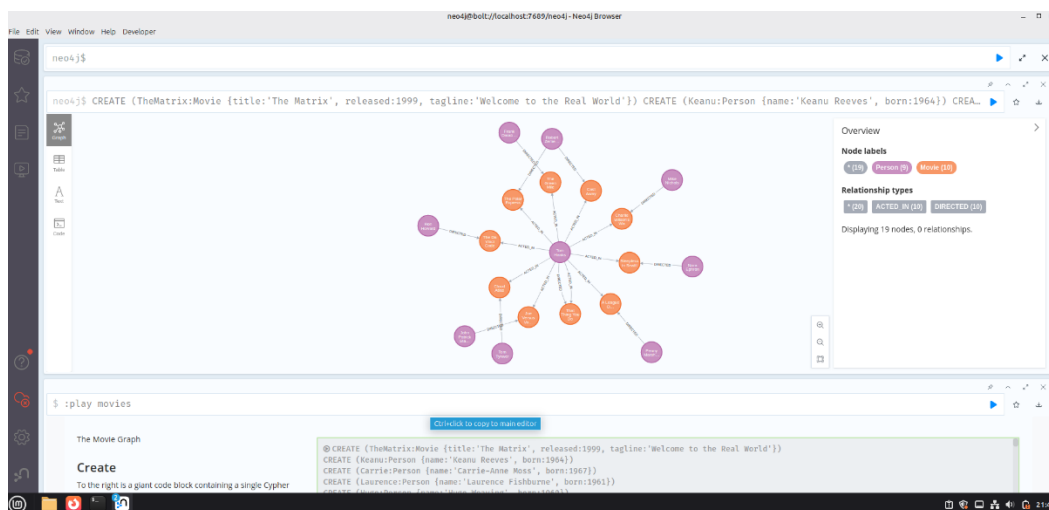


Figura 1. Grafo de prueba cargado con nodos Movie y Person.

Paso 2. Cambio al contexto administrativo

En Neo4j, la gestión de usuarios, roles y privilegios no se realiza sobre la base de datos de aplicación, sino sobre la base de datos especial **system**, que almacena los metadatos de seguridad de la instancia. Por ello, antes de ejecutar cualquier sentencia administrativa, es necesario cambiar el contexto de trabajo.

```
// Cambiar al contexto administrativo (base de datos de sistema)  
:use system
```

Paso 3. Creación del rol restrictivo

Se diseñó un rol denominado **lector_peliculas**, concebido bajo el PoLP: nace sin ningún privilegio y recibe únicamente lo necesario. En primer lugar, se le otorgó el privilegio de **conexión** (**ACCESS**) a la base de datos **neo4j**; sin él, los usuarios asociados no podrían siquiera establecer sesión contra ella. En segundo lugar, se aplicó una regla granular de lectura mediante **GRANT MATCH**, que equivale a combinar los privilegios de recorrido (**TRAVERSE**) y lectura de propiedades (**READ**).

El aspecto técnico clave es la cláusula **NODES Movie**: esta restringe el privilegio a **nivel de etiqueta de nodo**, de modo que la capacidad de lectura se limita exclusivamente a los nodos etiquetados como **Movie**. Cualquier otro elemento del grafo —en particular, los nodos **Person**— queda fuera del alcance del rol y, por tanto, aislado. El modificador **{*}** indica que se permite leer todas las propiedades de dichos nodos.

```
// 1. Crear el rol; por defecto no posee ningún privilegio  
CREATE ROLE lector_peliculas;  
  
// 2. Permitir la conexión a la base de datos de aplicación "neo4j"  
GRANT ACCESS ON DATABASE neo4j TO lector_peliculas;  
  
// 3. Conceder Lectura (MATCH) de todas las propiedades,  
// restringida únicamente a los nodos con la etiqueta Movie  
GRANT MATCH {*} ON GRAPH neo4j NODES Movie TO lector_peliculas;
```

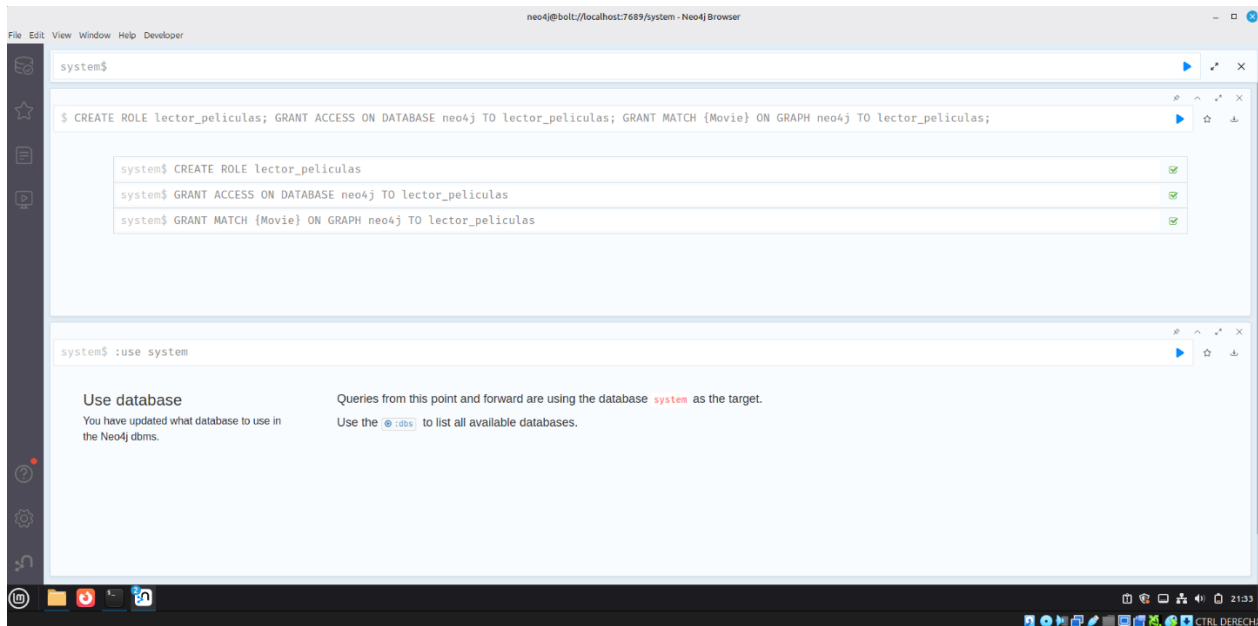


Figura 2. Creación del rol *lector_peliculas* y asignación de sus privilegios.

Paso 4. Creación de credenciales

Una vez definido el rol, se creó el usuario **pasante** con una contraseña inicial y se le vinculó al rol restrictivo. La opción **CHANGE NOT REQUIRED** evita que el sistema exija el cambio de contraseña en el primer inicio de sesión, lo cual resulta conveniente en un entorno de laboratorio controlado. Al asociar al usuario con **lector_peliculas**, este hereda exclusivamente los privilegios definidos en el paso anterior, sin permisos adicionales.

```
// Crear el usuario con su contraseña; no se exige cambio en el primer acceso
CREATE USER pasante SET PASSWORD 'pasante123' CHANGE NOT REQUIRED;

// Vincular al usuario con el rol restrictivo
GRANT ROLE lector_peliculas TO pasante;
```

Paso 5. Auditoría y pruebas de acceso

Para verificar el comportamiento del esquema de seguridad, se cerró la sesión del administrador mediante `:server disconnect` y se ingresó al sistema con las credenciales del usuario `pasante`. Posteriormente, se apuntó el contexto hacia la base de datos de aplicación con `:use neo4j`, desde donde se ejecutaron dos pruebas complementarias: una de acceso permitido y otra de acceso no autorizado.

```
// Cerrar la sesión del administrador
:server disconnect

// (Inicio de sesión con el usuario "pasante")

// Apuntar a la base de datos de aplicación
:use neo4j
```

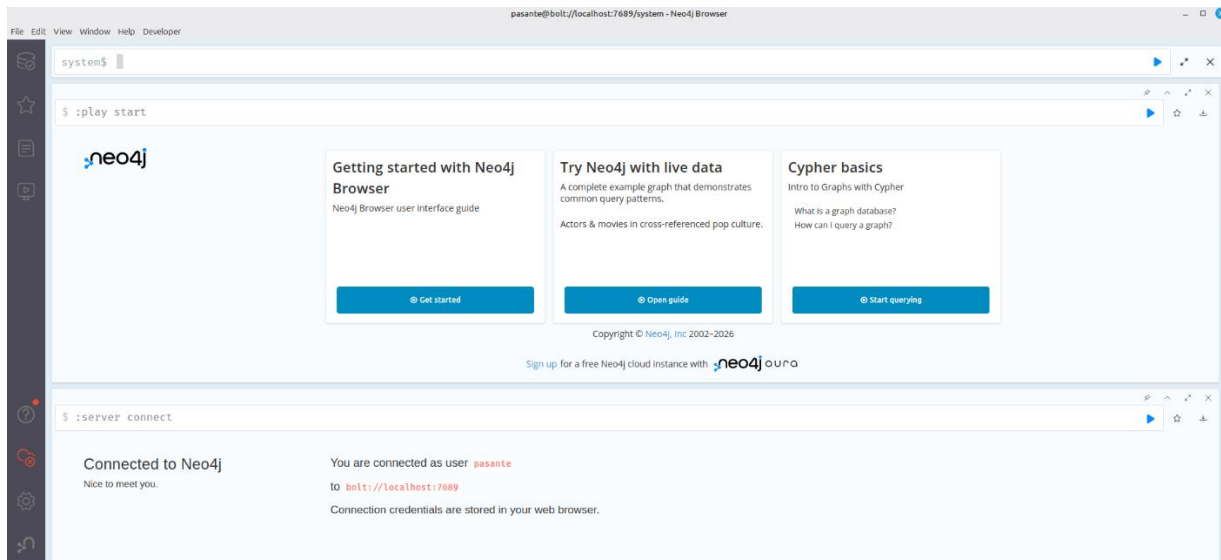


Figura 3. Inicio de sesión del usuario creado “pasante”

Prueba 1 (acceso permitido). Se consultaron los nodos **Movie**, sobre los cuales el rol posee el privilegio de lectura. El sistema procesó la solicitud exitosamente y devolvió los registros solicitados.

```
// Lectura de nodos autorizados por el rol  
MATCH (m:Movie) RETURN m LIMIT 5;
```

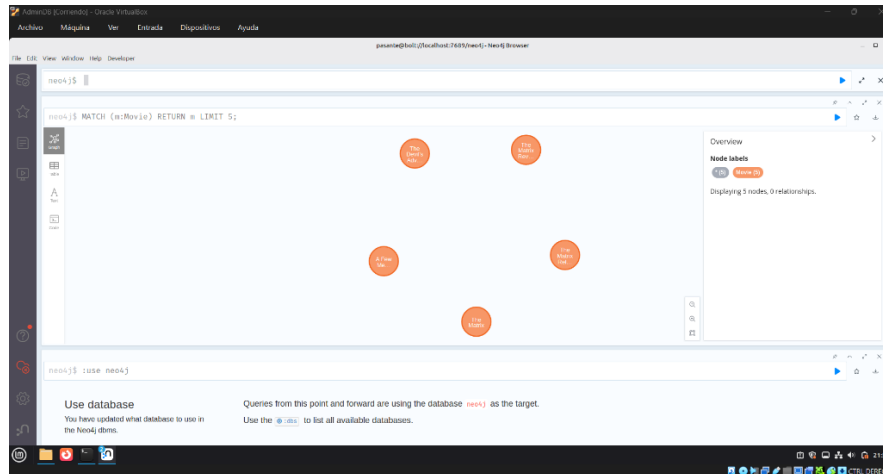


Figura 4. Resultado de la Prueba 1: acceso permitido a nodos *Movie*.

Prueba 2 (acceso denegado por PoLP). Se intentó consultar los nodos **Person**, sobre los cuales el rol no posee ningún privilegio. El motor de seguridad interceptó la consulta y no devolvió información (respuesta vacía o denegada), lo que demuestra la **invisibilidad** de los datos no autorizados: para el usuario restringido, dichos nodos simplemente no forman parte del grafo.

```
// Intento de Lectura de nodos NO autorizados por el rol  
MATCH (p:Person) RETURN p LIMIT 5;
```

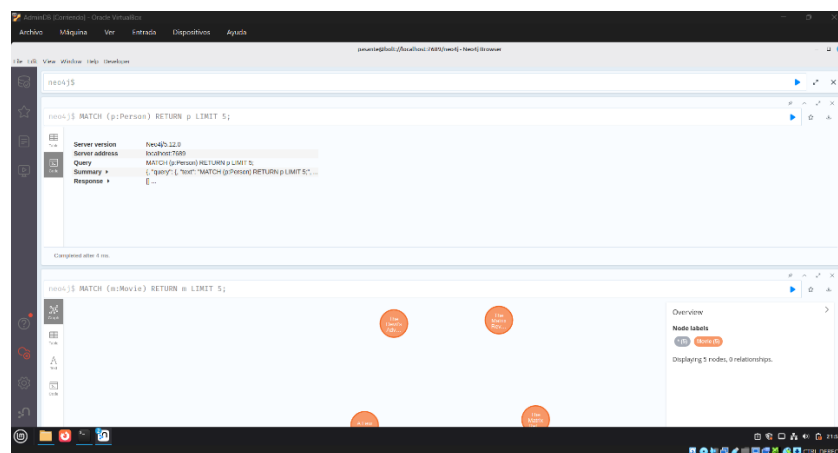


Figura 5. Resultado de la Prueba 2: acceso restringido a nodos *Person*.

4. Validación y Pruebas

La comprobación de los permisos se realizó contrastando el resultado esperado según el diseño del rol con el resultado efectivamente observado al operar como el usuario **pasante**. La siguiente matriz resume la verificación:

Prueba	Consulta ejecutada	Resultado esperado	Resultado obtenido
1	MATCH (m:Movie) RETURN m LIMIT 5;	Acceso permitido: privilegio MATCH sobre NODES Movie	Se devolvieron nodos Movie correctamente
2	MATCH (p:Person) RETURN p LIMIT 5;	Acceso restringido: ningún privilegio sobre nodos Person	Respuesta vacía o denegada; datos invisibles

Los resultados confirman que el rol **lector_peliculas** cumple con el Principio de Mínimo Privilegio: el usuario **pasante** puede ejecutar únicamente las operaciones de lectura necesarias sobre los nodos **Movie**, mientras que la información asociada a los nodos **Person** permanece completamente aislada. Asimismo, se comprobó que la restricción es aplicada por el motor de la base de datos y no depende de la lógica de ninguna aplicación cliente.

5. Conclusiones

La práctica demostró que Neo4j ofrece un modelo de seguridad lo suficientemente expresivo para trasladar el Principio de Mínimo Privilegio desde un enunciado teórico hasta una política verificable. La posibilidad de acotar un privilegio de lectura a una etiqueta de nodo específica mediante la cláusula **NODES** permite particionar lógicamente un grafo sin duplicar datos ni fragmentar la base en múltiples instancias, lo que preserva la integridad del modelo y, al mismo tiempo, protege la información sensible. En un modelo donde las relaciones son ciudadanos de primera clase, esta granularidad es especialmente valiosa, pues evita que un recorrido arbitrario exponga entidades que el usuario no debería conocer.

Desde la perspectiva del **gobierno de datos**, el aislamiento mediante roles aporta tres beneficios sustanciales. Primero, reduce el **riesgo de exposición**: una credencial comprometida o un error de un usuario de bajo privilegio queda confinado a un subconjunto acotado del grafo. Segundo, favorece la **auditabilidad y el cumplimiento normativo**, ya que los privilegios quedan declarados de forma explícita y centralizada en la base de datos **system**, lo que facilita demostrar quién tiene acceso a qué categoría de información. Tercero, promueve la **separación de funciones**: perfiles como pasantes, analistas o desarrolladores pueden trabajar con los datos que su labor requiere sin necesidad de recibir permisos administrativos ni acceso irrestricto.

En entornos de desarrollo, donde con frecuencia se opera con copias de datos reales y con múltiples colaboradores, la disciplina de crear roles específicos —en lugar de reutilizar cuentas con privilegios elevados— constituye una práctica de higiene de seguridad indispensable. Un diseño basado en roles reduce la fricción administrativa a medida que el equipo crece, evita la acumulación silenciosa de permisos y establece desde las primeras etapas del ciclo de vida del software una cultura de seguridad por diseño. Como líneas de mejora para un despliegue productivo, se recomienda complementar este esquema con una política robusta de gestión de credenciales, el uso de contraseñas únicas sujetas a rotación, la revisión periódica de privilegios y el registro de auditoría de los accesos.

En síntesis, la implementación de RBAC con granularidad de nodo en Neo4j no solo resuelve un requerimiento técnico puntual, sino que representa un componente esencial de una estrategia integral de gobierno de datos: aquella en la que la información sensible está protegida por defecto y el acceso se concede de manera deliberada, mínima y verificable.